GH-300

# Copilot Customization

## Prompt Files · Skills · Agents · Hooks

*Turn one Copilot into your team's Copilot — encoded, reviewed, and shared in the repo.*

Prompt files | Agent Skills | Custom Agents | Hooks | Awesome Copilot

Sandeep Soni · Deccansoft Software Services Pvt Ltd

*Deck 10 · GH-300 Certified GitHub Copilot Expert Training*

OVERVIEW

# The customisation landscape — four primitives, one repo

*Custom instructions teach Copilot HOW to write. These four primitives extend WHAT it can do, and as WHO.*

## Prompt files

Reusable, parameterised prompts saved as .prompt.md. Invoke as slash commands. Best for repeatable tasks (scaffold component, security audit, release notes).

## Agent Skills

Folders of instructions + scripts + resources. Loaded on demand when the description matches the task. Best for specialised workflows.

## Custom Agents

.agent.md personas with scoped tools, pinned models, and handoffs. Best for role-based work (Planner → Coder → Reviewer).

## Hooks

Pre / post-processing logic attached to a specific agent. Best for validation, auditing, and integrating downstream systems.

# Prompt Files — reusable prompts as slash commands

### Markdown format

.prompt.md files in .github/prompts/. Optional YAML frontmatter + Markdown body.

### Slash commands

Type / in chat → autocomplete shows your prompts → invoke instantly.

### Shared in repo

Live alongside your code. Reviewed in PRs. Same prompt across the whole team.

### Configurable

Pin agent mode, model, tools, description, and argument hints via frontmatter.

**Availability:** VS Code, Visual Studio, and JetBrains IDEs. github.com chat does not currently support slash-invoked prompt files.

# Anatomy of a prompt file — frontmatter + body

.github/prompts/reactform.prompt.md

```
---
agent: 'agent'
model: 'Claude Sonnet 4.6'
tools: ['githubRepo', 'codebase']
description: 'Generate a React form'
argument-hint: 'mention form name'
---

Your goal: generate a new React form
component based on our templates.

Requirements:
 * Use design-system components from
   [Form.md](../docs/design-system/Form.md)
 * Use react-hook-form for state
 * Use yup for validation
 * Always define TypeScript types

If the form name is missing, ask first.
```

## Frontmatter fields

**`agent`**

"agent" (agentic) or "ask" (Q&A)

**`model`**

Pin the LLM (GPT-5.2-Codex, Claude Sonnet 4.6, …)

**`tools`**

Array of tools / MCP servers to enable

**`description`**

Shown in the slash-command menu

**`argument-hint`**

Placeholder text for extra input

# Variables in prompt files — make them flexible

| | | |
|---|---|---|
| **Workspace** | | |
| | `${workspaceFolder}` | Path to the workspace / project root |
| **Selection** | | |
| | `${selection}` | Currently selected code in the editor |
| | `${selectedText}` | Text content of the current selection |
| **File** | | |
| | `${file}` | Path to the active file |
| | `${fileDirname}` | Directory of the active file |
| **Input** | | |
| | `${input:variableName}` | Custom input passed in via the chat input |

*Example: "Review the code in ${file} focusing on ${input:reviewFocus}" — flexible without rewriting the prompt.*

# Where prompt files pay off — six high-leverage use cases

### Scaffold components

Consistent React / Angular / Vue components from one template.

### Run & fix tests

Execute tests + fix failures with project context.

### Prepare pull requests

PR descriptions, changelogs, reviewer checklists.

### Security reviews

Standardised audit checklist applied across the team.

### Generate documentation

API docs, READMEs, ADRs from code + context.

### Override agent behaviour

Minimal-implementation plans, API mockups, dry-run prompts.

# Example — React form generator

reactform.prompt.md

```
---
agent: 'agent'
model: 'Claude Sonnet 4.6'
tools: ['githubRepo','search/codebase']
description: 'Generate a new React form'
argument-hint: 'form name + fields'
---

Generate a React form component using
the templates in #tool:githubRepo
contoso/react-templates.

Ask for the form name and fields if
they are not provided.

Requirements:
  * design-system components
  * react-hook-form for state mgmt
  * yup validation; TS types always
  * reusable validation schemas
```

## How it works

**Agent mode** — runs in full agentic mode; can create files, read repos, search.

**Model pinned** — consistent generations across the team.

**Tools enabled** — githubRepo + codebase search for context-aware output.

**Smart fallback** — asks for the form name and fields if missing.

**Standards enforced** — react-hook-form + yup + TS, every time.

# Example — REST API security review

securityreview.prompt.md

```
---
agent: 'ask'
model: 'Claude Sonnet 4.6'
description: 'REST API security review'
---

Perform a REST API security review
and provide a Markdown TODO list.

Check:
 * All endpoints protected by
   authentication + authorisation.
 * All inputs validated and sanitised.
 * Rate limiting / throttling in place.
 * Logging + monitoring for sec events.

Return the TODO grouped by priority
and issue type.  Cite line numbers.
```

## Key differences from Example 1

**Ask mode** (agent: ask) — analyses and reports rather than editing files.

**No tools needed** — uses only the codebase context the user provides.

**Structured output** — explicitly asks for a Markdown TODO grouped by priority + type.

**Team benefit** — every developer uses the same security checklist on every PR.

# Testing a prompt file safely before sharing

### 1 Start in Ask mode

Set agent: ask while iterating. No file writes; no surprises.

### 2 Sandbox the inputs

Test on a throwaway branch or a sample repo first. Never iterate against main.

### 3 Run the play button

In VS Code, use the editor "Play" button on the prompt file. Runs in a scratch chat.

### 4 Inspect the references

Open the References list in the response. Confirm which files / tools were actually used.

### 5 Diff outputs across models

Pin different models in turn (Claude / GPT) — compare. Pin the one that wins consistently.

### 6 Promote with a name

Rename from draft-foo.prompt.md to foo.prompt.md only when you trust it. Commit.

### Red flags before you commit

Tools that should never fire for this task are in the tools list — delete them. The prompt edits files when agent: ask is enough. The instructions reference docs / files that no longer exist. Outputs vary wildly across runs — your prompt is under-specified.

# Agent Skills — folders of instructions, scripts, and resources

*A specialised capability that loads on demand. Like a prompt file with attached files, scripts, and templates — and an open standard honored across AI tools.*

- Language / framework expertise
- Architecture rules
- Coding standards
- Security practices
- Testing strategy
- Cloud / infrastructure
- Performance expectations
- Doc generation pipelines

**Where they live**

`.github/skills/<skill-name>/SKILL.md` · `.agents/skills/<skill-name>/SKILL.md` · `.claude/skills/` (Claude Code compatible)

*Same folder structure across tools — write once, run anywhere AGENTS-style.*

# Creating an Agent Skill — directory + SKILL.md frontmatter

directory layout

```
.github/skills/
  webapp-testing/
    SKILL.md           # instructions
    test-template.js   # template
    examples/          # samples
      login-test.js
      search-test.js
```

SKILL.md

```
---
name: webapp-testing
description: Guide for testing
  web apps using Playwright.
  Use when asked to create
  or run browser-based tests.
---

# Web Application Testing
[Instructions, examples, guidelines]
```

## Skill body should include

- What the skill helps accomplish (one paragraph).
- When to use the skill (concrete triggers — Copilot uses these to decide relevance).
- Step-by-step procedures to follow.
- Examples of expected input / output — with the input wrapped in a clear delimiter.
- References to included scripts / resources (relative paths).

# How Copilot uses skills — discover, load, fetch resources

### 1 Discovery

Copilot reads the name + description from every skill's YAML frontmatter. Lightweight scan — cheap, runs always.

### 2 Loading

When your request matches a skill's description, Copilot loads the full SKILL.md body into context.

### 3 Resources

Additional files (scripts, templates, examples) are loaded ONLY when referenced. Keeps the token budget lean.

*On-demand loading keeps your context window efficient — only relevant skills consume tokens. Description quality is everything; write it to be matchable.*

# Skills vs Prompts vs Instructions — when to use which

| Feature | Agent Skills | Prompt Files | Custom Instructions |
|---|---|---|---|
| **Purpose** | Specialised capability | Reusable prompt as slash cmd | Coding standards always on |
| **Loading** | On demand (description match) | Explicit invocation | Always in context |
| **Content** | Instructions + scripts + files | Markdown + frontmatter | Markdown only |
| **Portability** | VS Code, CLI, cloud agent | VS Code, VS, JetBrains | VS Code + github.com |
| **Standard** | Open (agentskills.io) | VS Code / GitHub-specific | GitHub-specific |

**Use together:**

Custom Instructions for always-on standards (naming, formatting, architecture). Prompt Files for repeated team tasks (scaffold, audit). Agent Skills for specialised workflows (browser testing, IaC review) that load only when relevant.

# Agent-scoped hooks run scripts only while that agent is active

*Add a hooks map to a custom agent's .agent.md frontmatter to run commands at lifecycle events. They run in addition to workspace and user hooks. VS Code preview: Local harness only; needs chat.useHooks and a trusted workspace.*

**UserPromptSubmit** — Fires when you submit a prompt. Audit the request or add context.

**PreToolUse** — Fires before each tool call. Block it, ask for approval, or change the input.

**PostToolUse** — Fires after a tool succeeds. Run a formatter or validate the result.

**Stop** — Fires when the agent finishes. Runs as SubagentStop inside a subagent.

.agent.md · hooks

```
# .github/agents/bug-hunter.agent.md   (frontmatter excerpt)
hooks:
  PreToolUse:  [{ type: command, command: "node scripts/hooks/guard-tools.js" }]  # deny or ask first
  PostToolUse: [{ type: command, command: "./scripts/hooks/log-audit.sh" }]       # audit trail
```

# Hooks — where they earn their keep

### Compliance audit

PostToolUse logs every tool call to Splunk or Datadog: session, agent, tool name, inputs, and result.

### Secret & PII guard

PreToolUse inspects tool input and denies calls that would send keys, tokens, or customer data to fetch or MCP tools.

### Output validation

PostToolUse runs a JSON Schema check or lint pass on edited files and feeds failures back so the agent fixes them.

### Notification fan-out

Stop posts a Slack message when Bug-Hunter finishes and opens a ticket when it reports high severity.

### Destructive-command block

PreToolUse denies rm -rf, force pushes, and DROP TABLE, or returns "ask" so a person approves first.

### Style enforcement

PostToolUse runs Prettier or dotnet format on every file the agent edits, so diffs arrive formatted.

# awesome-copilot — community-curated starting points

## What it is

- github/awesome-copilot — community-contributed instructions, prompts, skills, and agents.
- Searchable + filterable index across hundreds of resources.
- Tools section for MCP servers and dev tooling.
- Learning Hub with guides and tutorials.

## How to use it well

- Start with a vetted prompt / skill, then adapt to your repo conventions.
- Treat it as a reference for HOW to write — not a paste-into-production source.
- Pin to a specific commit when you copy — community entries do drift.
- Contribute back: your domain-specific skill is someone else's starting line.

# Key takeaways

1

### Four primitives, one repo

Prompt files · Agent Skills · Custom Agents · Hooks. Layer with Custom Instructions (Deck 3) to teach Copilot HOW + WHAT + WHO.

2

### Prompt files = reusable slash cmds

.github/prompts/*.prompt.md. Frontmatter pins mode, model, and tools. Body sets the goal.

3

### Test before sharing

Ask mode → sandbox → play button → check references → diff across models → rename from draft.

4

### Skills load on demand

Discovery on description, body on match, resources on reference. Token-efficient by design.

5

### Custom Agents enforce least privilege

.agent.md persona with scoped tools — a planner that literally cannot edit code. Handoffs chain roles safely.

6

### Hooks enforce policy at run time

UserPromptSubmit · PreToolUse · PostToolUse · Stop in .agent.md frontmatter. Audit, block, validate, notify.

# Thank you.

*Bring your repo — we will encode its conventions together.*

Sandeep Soni · Deccansoft Software Services Pvt Ltd